

Note: This project taught me more about machine learning, deep learning, finance and my new found ability and talent to stare at the google collab's screen for a really really long time.

Due to multiple iterations and retries, several models with only marginal improvements were developed. For the sake of clarity and sanity, this report focuses solely on the final model.

Introduction and Problem Statement

Predicting the stock price is perhaps one of the oldest and the most studied problems in finance. The main challenge in doing so is that the market is driven by noise, human decisions, monetary policy, global news and so many other variables. It makes it almost impossible to account for all of this. This report details and tests the use of multiple models to find which proves to be the best for predicting NIFTY 50 price across two different market conditions.

Research question(s):

Primary

1. Which forecasting model (classical, machine learning, deep learning) most accurately predicts NIFTY 50 index closing prices using historical price data, technical indicators and macroeconomic factors.
2. To what extent can these models predict the daily directional moment of the NIFTY 50 index using the same historical price data, technical indicators and macroeconomic factors?

Secondary

1. Do these predictions look different for a market that is stable vs. volatile (Pre-Covid and Post-Covid)?

Data Overview

The primary dataset is NIFTY50 index from 2009-04-06 to 2024-12-30 pulled from yahoo finance (yfinance). For each day, we used the following: Open, High, Low, Close and Volume (OHLCV). Apart from this four macro features were pulled to provide a broader market context from the same source as listed in table (1&2) from the same source. Maco features like RBI repo rate and India 10-Year Bond Yield data were added initially, but were dropped from the final dataset because of inconsistencies.

For the sake of this project, I created two separate datasets (D1 and D2). Dataset 1 contained data up until 2019 (pre-covid) and D2 consisted of all 15 years data. The reason for this was to see and understand how the models perform in different conditions (pre and post Covid).

Two columns were created: log_close (the natural log of the Close price) and log_diff (the day-over-day log difference of the Close price, a proxy for daily returns). I also used shift(1),

which meant that every feature was shifted by one day before being used. This was done so that the model only sees past information, preventing data leakage.

Feature Engineering

In totality, there were 37 features that made it into the final model.

Technical features: Build using the OHLCV columns of the NIFTY 50 Index.

Feature	Description	How It Was Calculated
lag_1	Close price from 1 day ago	Close.shift(1)
lag_2	Close price from 2 days ago	Close.shift(2)
lag_3	Close price from 3 days ago	Close.shift(3)
lag_5	Close price from 5 days ago	Close.shift(5)
lag_10	Close price from 10 days ago	Close.shift(10)
lag_21	Close price from 21 days ago	Close.shift(21)
sma_5	5-day simple moving average	Rolling mean of past 5 Close prices, shifted 1 day
sma_10	10-day simple moving average	Rolling mean of past 10 Close prices, shifted 1 day
sma_20	20-day simple moving average	Rolling mean of past 20 Close prices, shifted 1 day
sma_50	50-day simple moving average	Rolling mean of past 50 Close prices, shifted 1 day
ema_10	10-day exponential moving average	Exponentially weighted mean over 10 days, shifted 1 day
ema_21	21-day exponential moving average	Exponentially weighted mean over 21 days, shifted 1 day
sma_5_20_ratio	Ratio of 5-day to 20-day SMA	sma_5 / sma_20; values above 1 suggest short-term strength
sma_10_50_ratio	Ratio of 10-day to 50-day SMA	sma_10 / sma_50; used to detect trend crossovers
ema_sma_ratio	Ratio of EMA to SMA	ema_10 / sma_10; measures how responsive recent prices are
momentum_5	Price change over 5 days	Close.shift(1) minus Close.shift(6)
momentum_10	Price change over 10 days	Close.shift(1) minus Close.shift(11)
momentum_21	Price change over 21 days	Close.shift(1) minus Close.shift(22)
return_1d	1-day log return	log_diff.shift(1)

Feature	Description	How It Was Calculated
return_1d	1-day log return	log_diff.shift(1)
return_5d	5-day cumulative log return	Sum of log_diff over past 5 days, shifted 1 day
return_10d	10-day cumulative log return	Sum of log_diff over past 10 days, shifted 1 day
volatility_10	10-day rolling standard deviation of log returns	Std of log_diff over 10 days, shifted 1 day
volatility_20	20-day rolling standard deviation of log returns	Std of log_diff over 20 days, shifted 1 day
rsi_14	Relative Strength Index over 14 days	100 minus (100 divided by (1 plus average gain divided by average loss)), shifted 1 day
bb_upper	Upper Bollinger Band	20-day SMA plus 2 times 20-day standard deviation
bb_lower	Lower Bollinger Band	20-day SMA minus 2 times 20-day standard deviation
bb_width	Width of Bollinger Bands as a fraction of mid band	(bb_upper minus bb_lower) divided by sma_20
bb_pct	Position of price within the Bollinger Bands	(Close minus bb_lower) divided by (bb_upper minus bb_lower)
macd	MACD line	12-day EMA minus 26-day EMA, shifted 1 day
macd_sig	MACD signal line	9-day EMA of the MACD line
macd_hist	MACD histogram	macd minus macd_sig
atr_14	Average True Range over 14 days as a fraction of Close	Rolling mean of True Range over 14 days divided by Close, shifted 1 day

Figure (1): Technical features

Macroeconomic features: Additional market context as pulled from yahoo finance.

Feature	Description	How It Was Calculated
macro_niftybank	NIFTY Bank index Close price	Close price of ^NSEBANK, shifted 1 day
macro_gold	Gold futures Close price	Close price of GC=F, shifted 1 day
macro_usdinr	USD to INR exchange rate	Close price of INR=X, shifted 1 day
macro_crude	Crude oil futures Close price	Close price of CL=F, shifted 1 day

Figure (2): Macroeconomic features

EDA (Exploratory Data Analysis)

Before going into the modelling part we understand the data itself. We perform an exploratory data analysis to understand the trends, distributions, return characteristics, correlations between variables, and other statistical properties.

Price Trends and other technical indicators

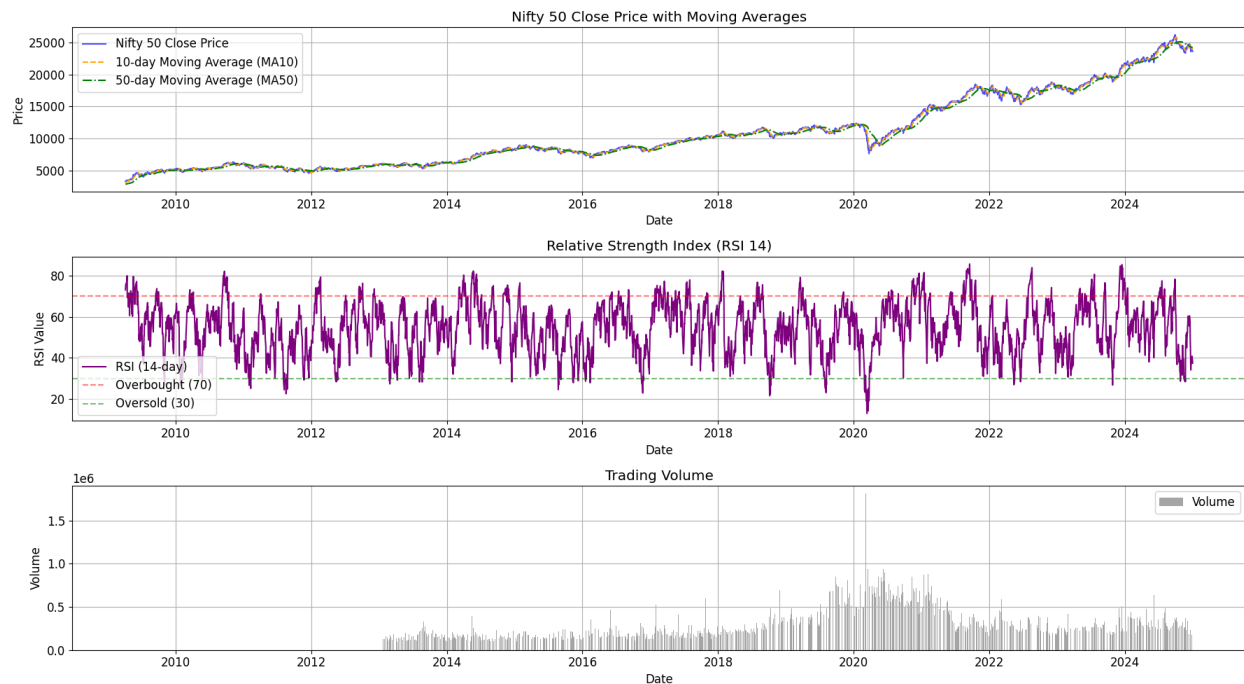


Figure (3) EDA Plots
Nifty 50 Close Price, RSI, Trading Value (2009-24)

For the close price, we can see an upward trend, the index started near 4,000 (around 2010) and crossed 24,000 by the end of 2024. This is due to the economic growth of the country. In the context of this project, this drastic change from 4,000 to 24,000 will be a challenge during the train/test split for model training. There is only one major spike, the COVID crash around 2020, where the index fell drastically before recovering back to normal.

The RSI is used to measure speed and magnitude of price changes over the years. It looks at the previous 14 days and calculates the ratio of average gains to average losses over that specific window (between 0-100), which acts as a baseline. Unlike price, this is stationary and helps models understand if the current price is rare and also helps understand the likelihood of what happens tomorrow by measuring if the current trend is gaining or losing strength.

Trading volume was relatively low, with an increase between 2020-2022 due to aggressive selling and buying.

Rolling Volatility

The 20-day rolling volatility chart shows how much daily returns fluctuated over a 20-day period.

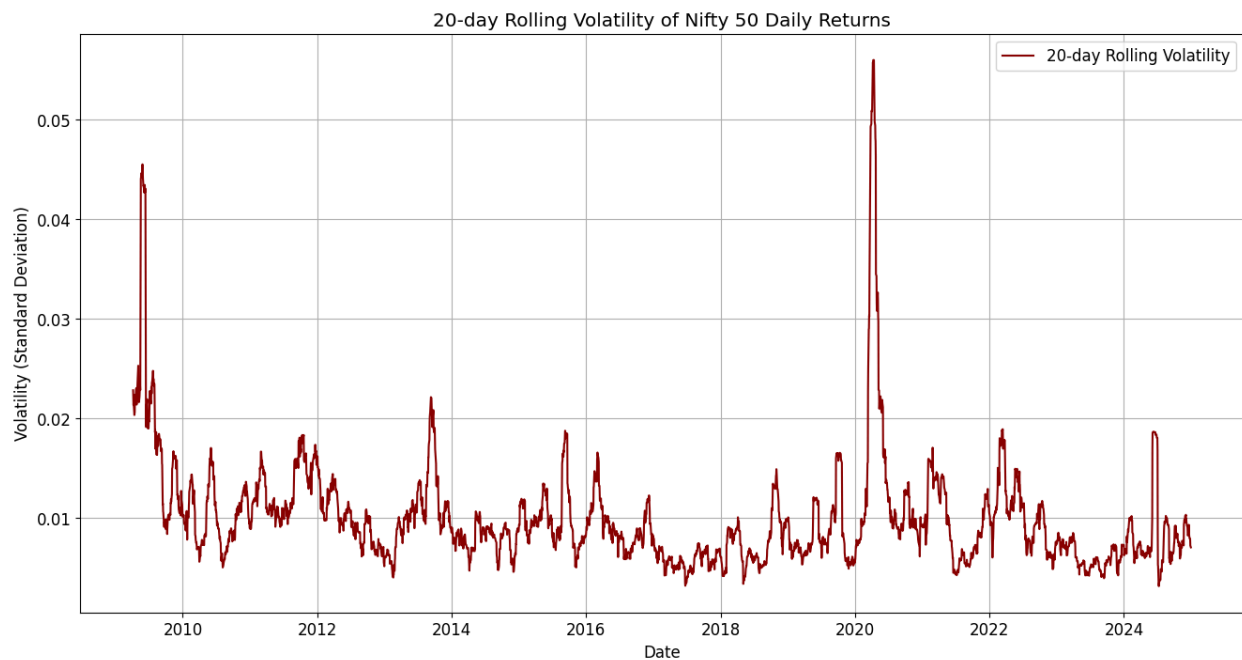


Figure (4) Rolling Volatility over the years (2009-2024)

Over the 15 year time frame, we see two major peaks. The first peak coincides with the global financial crisis where the volatility reaches around 5%. The second peak is even more dramatic due to the COVID era, where volatility reached around 6% before coming back to normal historical levels.

Distribution of Daily Returns

It is the actual distribution compared to the normal distribution curve.

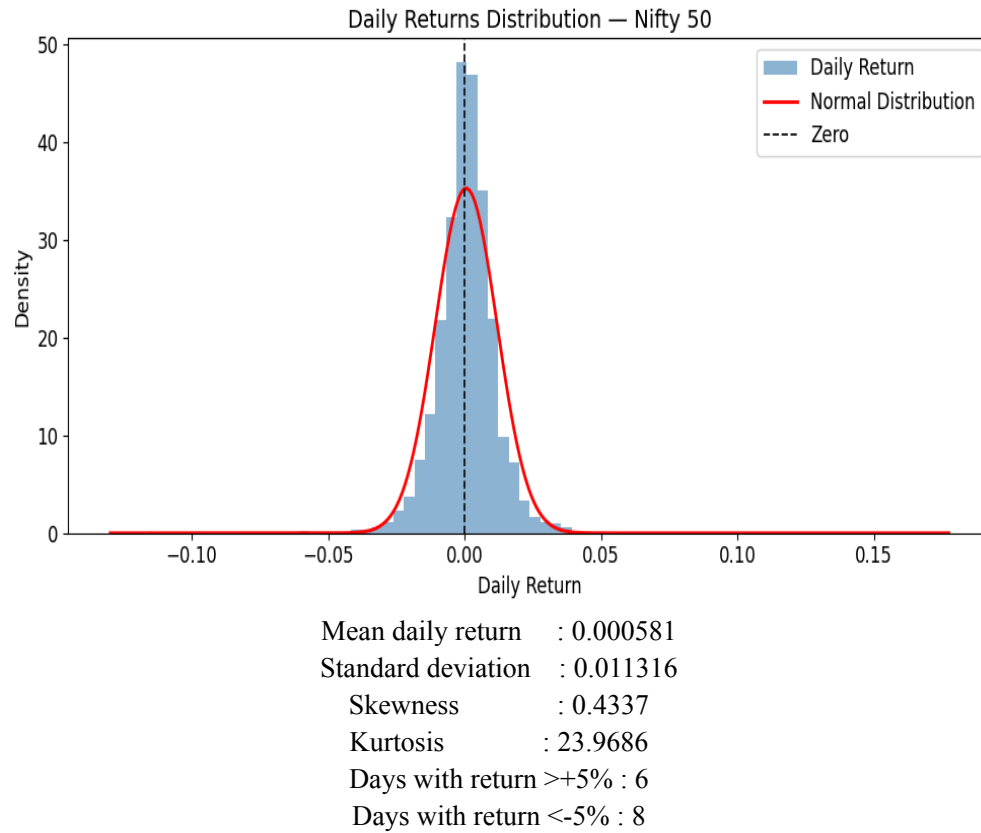


Figure (5): Daily Returns Distribution

Most of the bars are clustered around zero. This basically suggests that the Nifty 50 only stays between -1 to +1 or 2%. This is a leptokurtosis graph, which indicates higher volatility and risk. In simpler terms, extreme gains or losses happen more often than what a standard mathematical model would expect.

Correlation Analysis

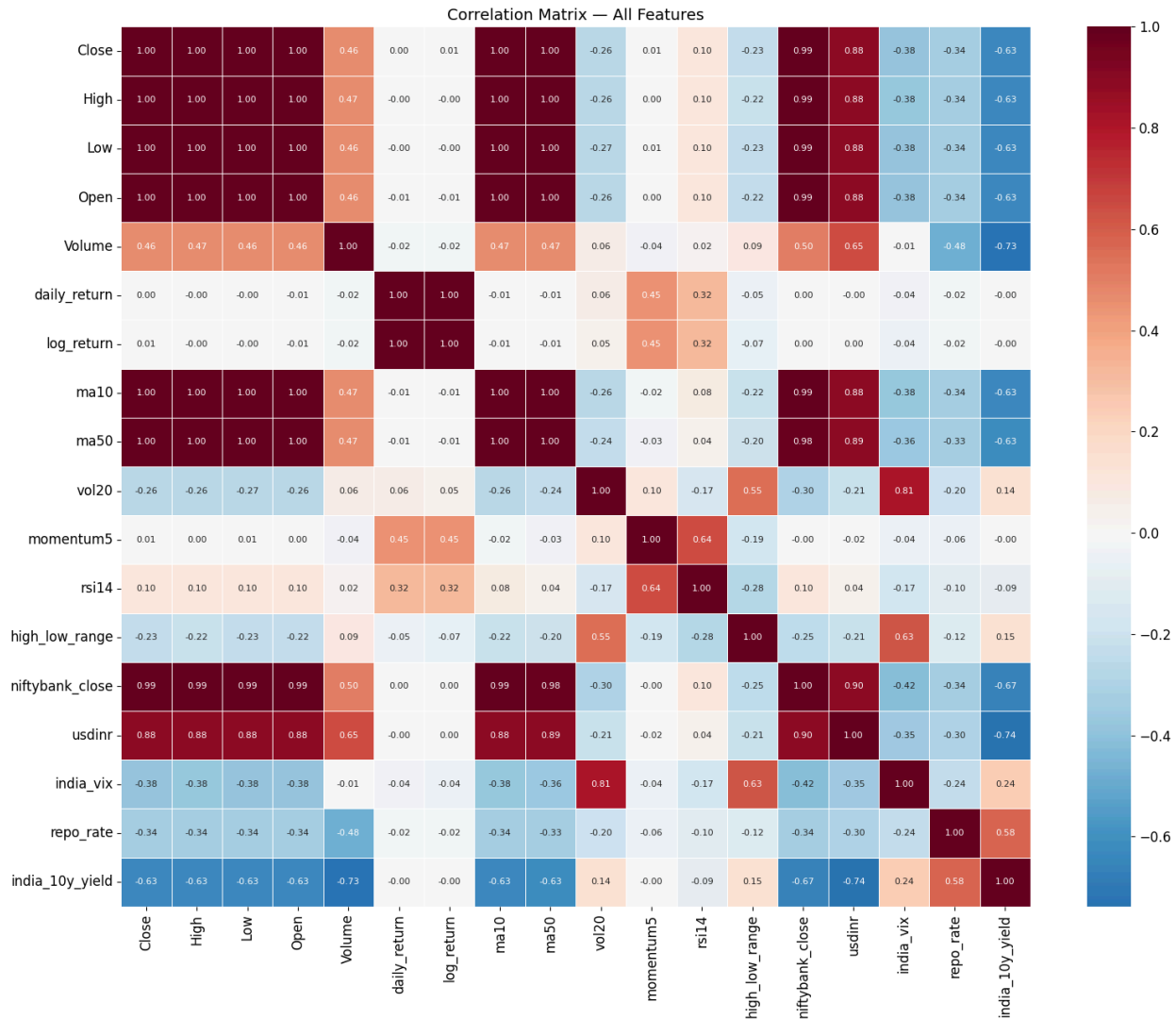


Figure (6): Correlation Plots

Unsurprisingly, there is a near perfect correlation between OHLCV, MA(10) and MA (50) which makes sense because these are engineered features and they follow the actual price very closely. This is not a problem for modeling because we are using lagged features for each. The NIFTY bank close price is very strongly correlated with the NIFTY 50 (0.9874) which again makes sense because a large portion of NIFTY 50 is made up of bank stocks.

India VIX shows a moderate negative correlation of (-0.38) and the USD/INR exchange shows a positive correlation of (+0.88).

Stationarity Tests

Stationarity in time series is characterized by a constant mean and variance over time. Many models require the inputs to be stationary because they assume that the patterns don't shift with time. I used two statistical tests: Augmented Dickey-Fuller (ADF) and KPSS.

```
=====
Series: Close
ADF Statistic : 0.9890 | p-value: 0.9941
KPSS Statistic: 8.5544 | p-value: 0.0100
Verdict       : NOT STATIONARY

=====
Series: daily_return
ADF Statistic : -14.5565 | p-value: 0.0000
KPSS Statistic: 0.0620 | p-value: 0.1000
Verdict       : STATIONARY

=====
Series: log_return
ADF Statistic : -14.5218 | p-value: 0.0000
KPSS Statistic: 0.0529 | p-value: 0.1000
Verdict       : STATIONARY
=====
```

Figure (7): Results of ADF and KPSS tests

The raw NIFTY 50 Close price failed both tests and were concluded to be stationary. Daily Return and log return on the other hand were both stationary. This becomes important in the future when I go into modeling.

ACF (autocorrelation) and PACF (partial autocorrelation) of Log Returns

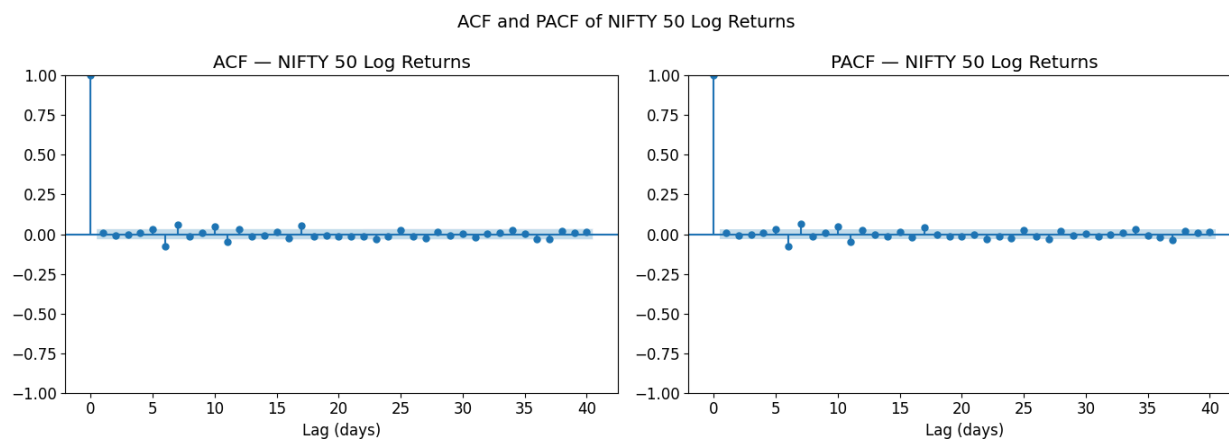


Figure (8) ADF and PACF plots

Perhaps the most important part of the project. In simple words ACF and DCF are diagnostic tools that help us understand if the market has a 'memory' or if it is random, or at least close to

random, in nature. They measure the total correlation between the stock's current return and its past returns at a specific time period. Each dot represents a lag (lag 1 = yesterday, lag 2= day before) and the height shows how much the price of yesterday affects today. Almost all our data is essentially within the blue zone, and therefore is just noise. There is no pattern that exists. **It's a random walk.**

Additionally, since there is no meaningful pattern within the price changes themselves, I rely on the macro and the external features for modelling later in this project.

Note: This introduced me to the concept of **Efficient Market Hypothesis** which says that stock prices always reflect all available information which makes it impossible to consistently beat the market.

Overview Of the Models:

1. ARIMA AND SARIMA

ARIMA is a classical time series model that uses three components: the past values of the series (autoregressive), past errors the model made (moving average) and the difference between values to remove trends. SARIMA is an extension of ARIMA that in addition to it captures the seasonal component as well. The model was fitted onto the data using the `pm.auto_arima` with `seasonal parameters=True`. This function basically looks for the best combination of the `p,d,q / P,D,Q,S` for SARIMA.

Note: It is important to note that this was fitted on/fitted to predict the log of closing price but since we take the first difference, $d=1$. The mathematical equation basically turns into the log return.

Equations:

Step 1: Log Transformation

$$\text{Log Price} = \ln(P_t)$$

Step 2: Differencing: The model subtracts the previous day's log price from the current day's log price.

$$\Delta \ln(P_t) = \ln(P_t) - \ln(P_{t-1})$$

On rewriting the equation this becomes the log return.

$$\ln(P_t) - \ln(P_{t-1}) = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

So, by differencing we successfully made the model stationary, but as the ACF and PACF plots showed earlier, the log return is random. So because of stationarity, we exposed the “random walk” nature of NIFTY 50. For this project the most optimal order was chosen for dataset 1 and dataset as:

best order: (2, 1, 0) seasonal: (2, 1, 0, 5)

2. Linear Regression

Perhaps the simplest predictive model. It assumes that the target is a linear combination of its input features. In this project, we performed a multiple linear regression with the entire set of 37 features, which were chosen with the help of feature importance. This model in our project serves as a benchmark.

3. Random Forest

A random forest model takes a large collection of decision trees from a random set of training data and features, makes individual predictions and then averages their individual predictions. The averaged results tend to be much more stable and less likely to overfit.

Initially, for this project I trained the model on the engineered features to predict a scaled version of next day's closing price. Hyperparameter tuning was used (number of trees, tree depth minimum samples per leaf) were also selected using the `RandomisedSearchCV` function.

The results were pretty bad. The reason can be attributed to the train/test split and the extrapolation (ability to estimate values outside of training data) of stock prices. The prices the model learned on the training data were very different from the ones it was tested on.

4. XGBOOST

XGBoost, unlike random forest, builds trees one after the other instead of independently. It focuses on correcting the errors with every new tree. This process is sequential and called gradient boosting. Since it was trained on the same features to predict the same target as Random Forest, the results were underdelivered.

5. Prophet

This model was developed by Facebook and it works by breaking the time series into three pieces: a flexible trend, seasonal patterns, and effect of holidays. It looks at the entire 15-year history of NIFTY 50 as a single shape and decomposes it into the three functions as mentioned. The equation is as follows:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

This was also applied directly on the scaled closing prices. This model also utilized the macrofeatures heavily.

Note: I initially tested all my models to predict the scaled version of the close price. But due to the reasons mentioned before, they all performed very poorly. So, instead of this, I switched to using **log-returns** (the day-over-day % change in log price) and then reconstructed the original price back from this to test them on the metrics. This solved the extrapolation problem because unlike prices, +1% returns look the same to the model. I will detail the results comparing both the situations later in the report.

6. Neural Prophet (Return-based)

Neural Prophet is an extension of Prophet which preserves the macro strengths of prophet (trends, cycles etc.) and introduces the ability of Neural Networks to capture more nuanced micro patterns. The AR-Net looks beyond the simple linear correlations and tries to learn from more complex patterns.

$$y_t = g(t) + s(t) + AR-Net + c^T r_t + \epsilon_t$$

Walk Forward Strategy

An important part of this project was to apply a ‘Walk-Forward Strategy.’ in traditional ML we split the data once (ex. 80/20 train-test) but this becomes risky for stock prediction particularly because the rules of the market are very dynamic in nature. So a model trained on 2010-2012 data will always fail to forecast the high growth era of 2023-2024. Here is how it works:

Phase (1): train on years 1-5 and predict the first 60 days of year 6.

Phase (2): take those 60 days as training set to predict the next 60.

The Model eventually will consume the entire dataset over 15 years.

The architecture is summarised below:

Component	Configuration
Target variable	Log-return: $\log(P_t/P_{t-1})$
Autoregressive lags (n_lags)	60 trading days
AR-Net hidden layers (ar_layers)	[64, 32] neurons
Trend type	Piecewise linear (growth = 'linear')
Yearly seasonality	Enabled (Fourier terms)
Weekly seasonality	Enabled (Fourier terms)
Loss function	Mean Squared Error (MSE)
AR regularisation	Enabled
Trend regularisation	Enabled
External regressors	1-day return, 5-day return, scaled RSI, volume z-score
Retraining strategy	Walk-forward, expanding window, every 60 days

Figure (9): Neural Prophet Architecture

7. LSTM (Long Short Term Memory Network)

LSTM is a type of neural network that is extremely beneficial for time series forecasting. Standard neural networks (ANN, CNN) treat each data point as independent, whereas LSTM carries a memory from one time step to the another. It controls the memory through a three gate mechanism: 1. Forget gate (decides on which information to keep which to erase) 2. An input gate (decides which information to store) 3. Output layer (decides what to pass onto the next layer). Like Neural Prophet, this was also trained on the log returns rather than the absolute price. A lookback window of 60 days was used. It basically allows the model to look at a fixed block of time instead of the entire history. For instance, to predict day 61 prices, it would analyse the features like RSI, Returns, Volume for days 1-60. This makes sure the decisions are relevant.

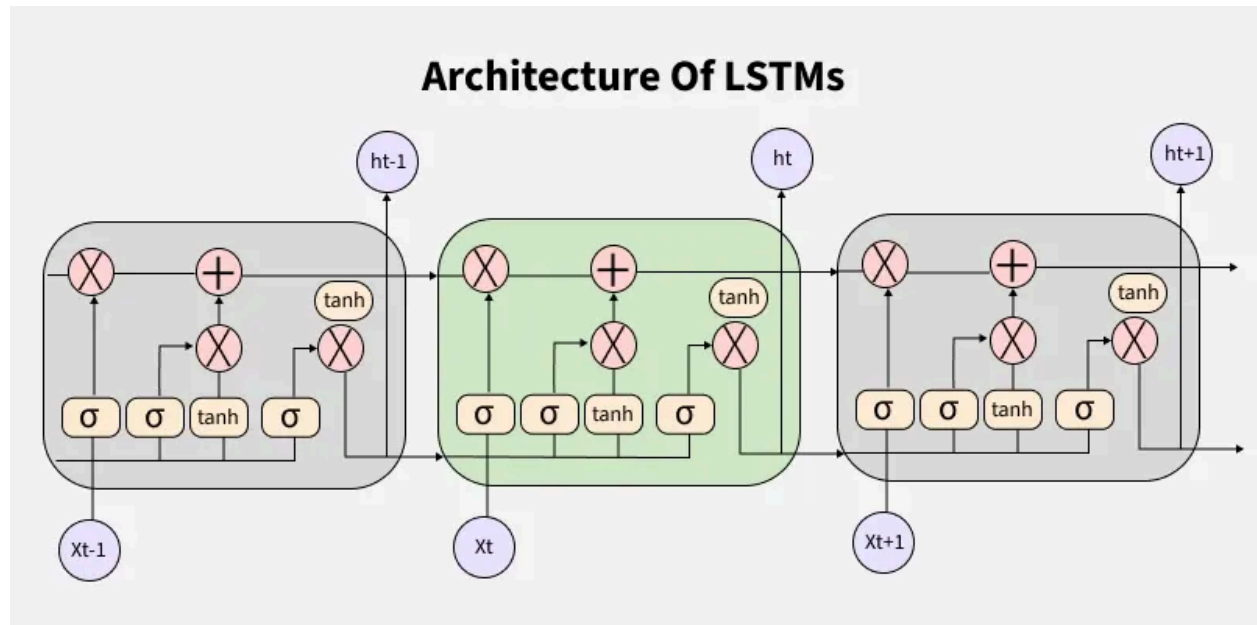


Figure (10): LSTM Model Architecture

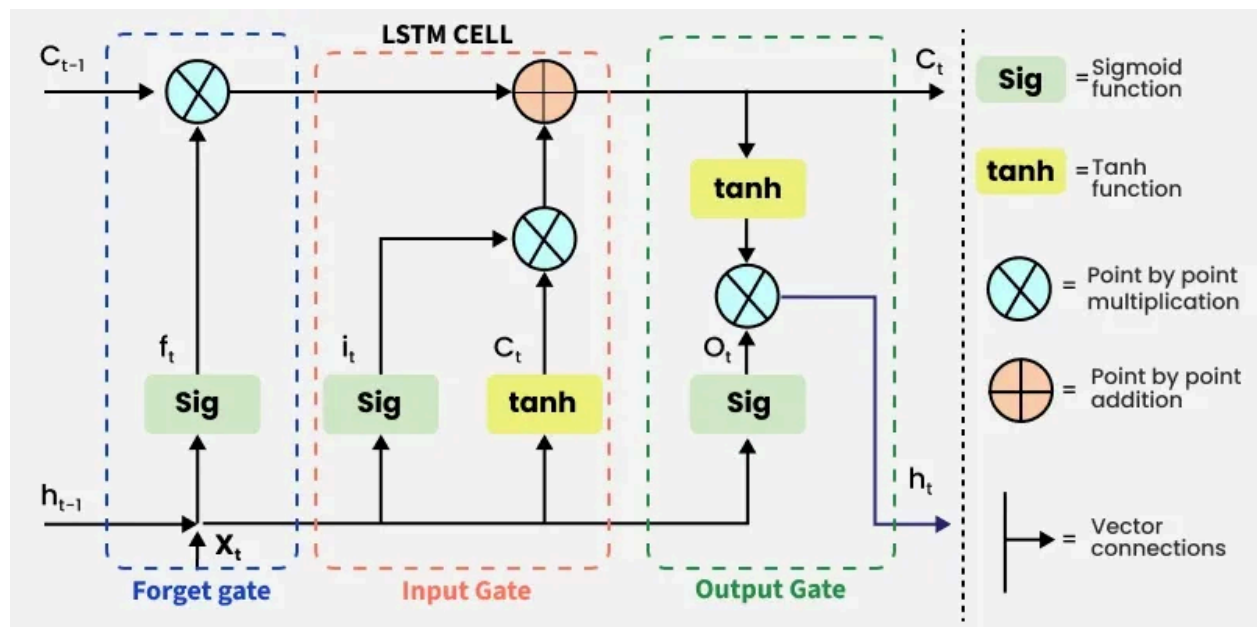


Figure (11): Working of an LSTM Model

Figure 1&2: Structure of an LSTM Network. Adapted from *Introduction to Long Short-Term Memory*, by GeeksforGeeks2023 (<https://www.geeksforgeeks.org/deep-learning/deep-learning-introduction-to-long-short-term-memory/>).

For the project this was the model architecture:

Layer	Type	Units / Details
Input	Sequence	Shape: (60 time steps, number of features)
LSTM Layer 1	Unidirectional LSTM	128 units, returns full sequence, L2 kernel
LSTM Layer 2	Unidirectional LSTM	64 units, returns full sequence, L2 kernel
LSTM Layer 3	Unidirectional LSTM	32 units, returns final hidden state, L2 kernel
Output	Dense (fully connected)	1 unit, predicts scaled log-return
Loss function	Mean Squared Error (MSE)	Suited to return prediction
Optimiser	Adam	With cosine learning rate schedule and
Gradient clipping	Enabled	Prevents exploding gradients on noisy
Early stopping patience	35 epochs	Higher than usual to account for noisy
Target variable	Scaled log-return	Converted back to price after prediction
Retraining strategy	Walk-forward, expanding window,	

Figure (12): LSTM architecture

Note: An initial question that I struggled with was to justify using models like LSTM which were built for memory even though the returns, as plotted earlier, were completely random. **Insight:** I then understood that ACF and PACF plots measure the Linear Correlation while the LSTM model looks for non linear patterns. Neural Networks (somewhat) are multivariate and have contextual intelligence, which basically means if given the right macroeconomic and technical indicators, they can understand the context extremely well.

Model Performance and Results

I used four metrics to assess and compare each of my models. All these metrics basically help to quantify the difference between actual vs predicted price. (a) MAE is the average of all individual predictions. An MAE of 88 indicates that the model is off by an average of 88 NIFTY points per day (b) RMSE squares the error before averaging them, then takes the square root. If the model has significantly worse RMSE compared to MAE, it is not great at dealing with outliers (C) MAPE converts error into percentage. Lastly R square is used to measure the proportion of variance in the market that the model's feature successfully explained.

Approach 01

Initial Results: All of these models were trained on the absolute (scaled) version of close price. Evidently so, all the machine learning algorithms (Random forest, XG boost, prophet) and Deep Learning models (LSTM, Neural Prophet) perform horribly.

===== MODEL COMPARISON =====					
	Model	RMSE	MAE	MAPE (%)	R ²
	ARIMA D1	74.32	56.09	0.543	0.9877
	ARIMA D2	176.51	129.02	0.668	0.9961
	Baseline D1	74.38	56.14	0.543	0.9877
	Baseline D2	176.50	128.95	0.667	0.9961
	LSTM D1	455.96	405.10	3.825	0.4725
	LSTM D2	2699.37	2334.26	11.125	0.1052
	NeuralProphet D1	313.03	237.60	2.333	0.7825
	NeuralProphet D2	465.68	361.38	1.873	0.9730
	Prophet D1	1821.66	1738.57	16.644	-6.3645
	Prophet D2	1513.48	1172.74	5.606	0.7153
	Random Forest D1	1623.62	1479.70	14.022	-4.8503
	Random Forest D2	3139.66	2066.03	9.141	-0.2252
	SARIMA D1	87.42	67.77	0.657	0.9830
	SARIMA D2	217.98	162.67	0.835	0.9941
	XGBoost D1	1705.93	1566.61	14.866	-5.4585
	XGBoost D2	3175.62	2071.19	9.125	-0.2534

Figure (13)

Model Comparison: dataset 1 and 2: Models ranked from the best to worst

Note: We concluded that NIFTY 50 was a random walk earlier in the report. So mathematically, the best approach to minimize any error is to simply predict: $Price(t)$ is $Price(t-1)$.

Interpretations

a) Baseline as the “best model”

The baseline model wins on paper because it essentially predicts that tomorrow’s price will be the same as today’s. In a financial time-series model, $Price(t)$ and $Price(t-1)$ are almost identical to each other. So simply copying that forward results in a very low RMSE, and a very high R square value. The model is not predicting any trends but just simply using the lag function here. Another important point to note is that the baseline model only “wins” for D1 (dataset 1) which only consists of data up until Pre-Covid (2019) when price movements were more consistent.

ARIMA and Baseline model both have a very similar, nearly identically RMSE, MAE, and MAPE value. They model understood that the most optimal path is similar to the previous day’s price.

b) Failure of ML and DL models

As stated earlier, ML models cannot extrapolate. XGboost, Random Forest, Prophet all show a negative R square value and very high MAPE values. The DL models likely struggled with the non-stationary nature of absolute prices.

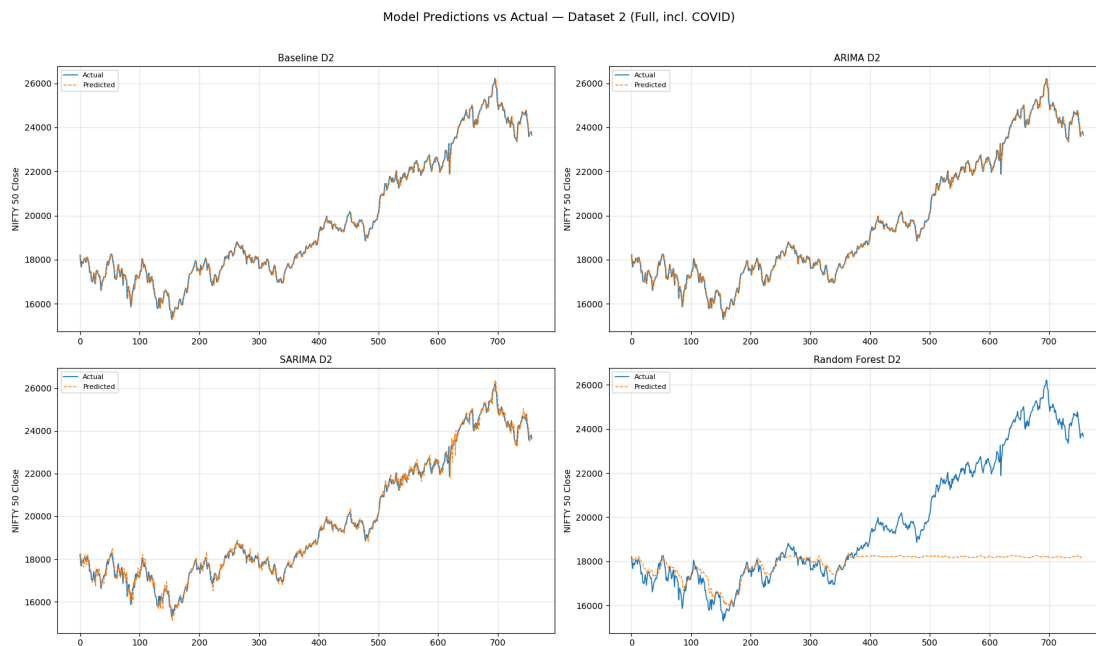


Figure (13a): Performance of on Dataset 2 Baseline, ARIMA, SARIMA, Random Forest

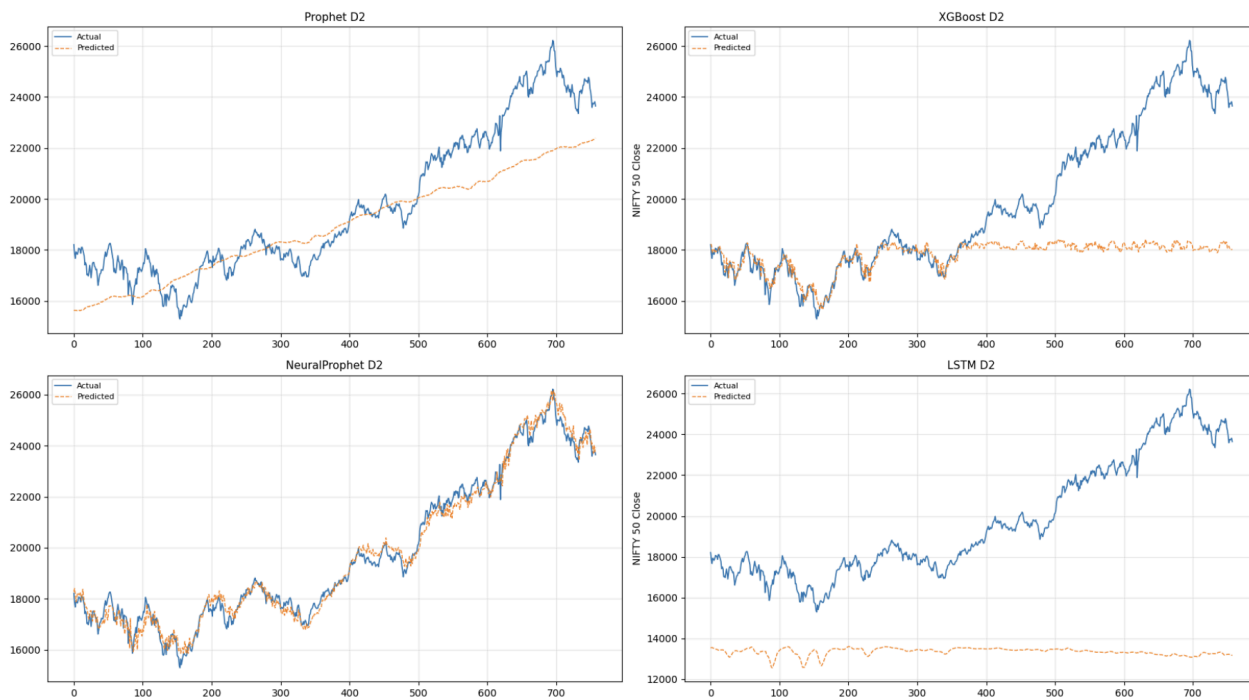


Figure (13b): Performance of on Dataset 2 Prophet, XGboost, NeuralProphet, LSTM

Approach 02:

Again to re-iterate the reason it did not work was because stock prices grow over time a model trained on older price data will always struggle with newswire prices. The scale was never constant. We switch our approach and analyse the **log of returns** instead of absolute prices which is better suited for our ML and DL models. This new change was implemented across all the models which included Random forest, XGBoost, Prophet, Neural Prophet and LSTM. The results are as follows.

D1 (Pre-COVID)						
Rank		Model	RMSE	MAE	MAPE (%)	R ²
1		Always Up Baseline D1	75.0	56.5	0.546	0.9868
2		Baseline D1	74.8	56.7	0.548	0.9869
3	LSTM	Improved (returns) D1	74.8	56.6	0.548	0.9869
4		LSTM Walk-Forward D1	75.2	56.9	0.550	0.9868
5		XGBoost D1	75.3	57.1	0.552	0.9867
6		Random Forest D1	79.7	63.1	0.609	0.9851
7		SARIMA D1	84.5	63.6	0.615	0.9833
8		Prophet D1	98.6	72.4	0.695	0.9773
9		Linear Regression D1	102.0	82.9	0.803	0.9757
10		NeuralProphet D1	249.7	197.5	1.926	0.8542
11		LSTM D1	1745.7	1608.8	15.261	-6.1281

D2 (Full)						
Rank		Model	RMSE	MAE	MAPE (%)	R ²
1	LSTM	Improved (returns) D2	176.2	128.4	0.664	0.9962
2		Always Up Baseline D2	176.6	128.4	0.664	0.9961
3		XGBoost D2	176.7	128.4	0.665	0.9961
4		Baseline D2	176.3	128.7	0.665	0.9962
5		LSTM Walk-Forward D2	177.2	129.0	0.667	0.9961
6		Random Forest D2	181.3	131.6	0.682	0.9959
7		Linear Regression D2	188.4	141.2	0.722	0.9956
8		SARIMA D2	207.7	153.5	0.789	0.9947
9		Prophet D2	231.4	177.3	0.882	0.9934
10		NeuralProphet D2	457.3	358.7	1.856	0.9741
11		LSTM D2	7955.5	7481.6	36.785	-6.8306

Figure (14): Full Model Comparison for Nifty 50 Forecasting (Improved)

Dataset 1

Always up baseline wins yet again on dataset 1. It simply just predicts a 0.1% rise everyday and naturally aligns with how the market moved during this period. Dataset was a smooth and consistent bull market (pre-2019) so simpler models were able to perform well.

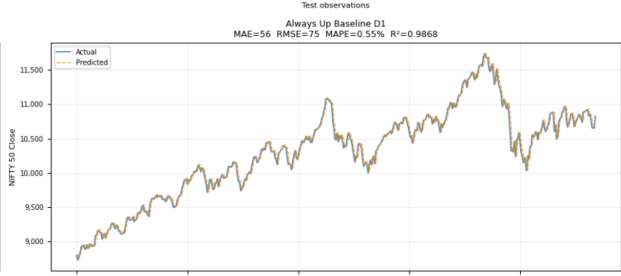
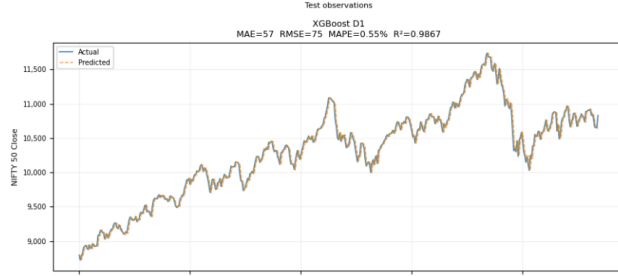
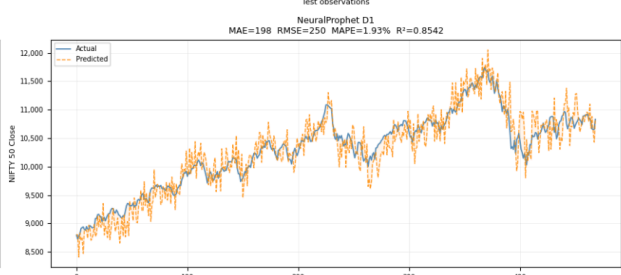
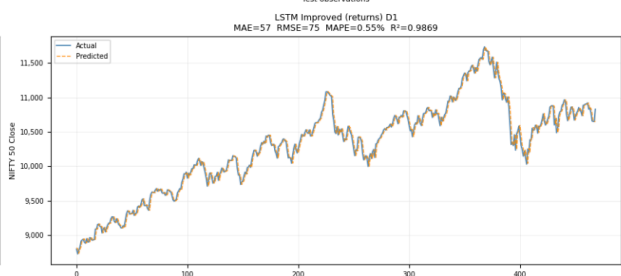
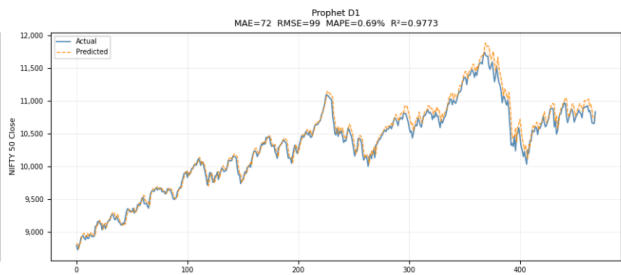
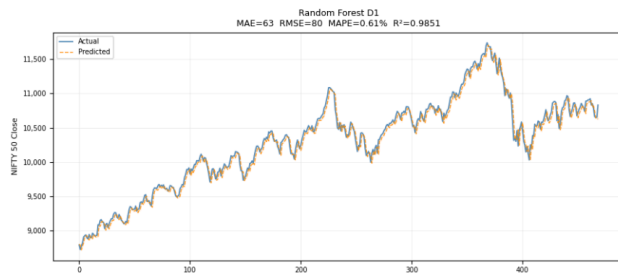
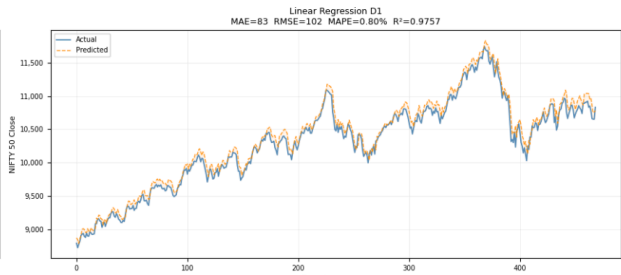
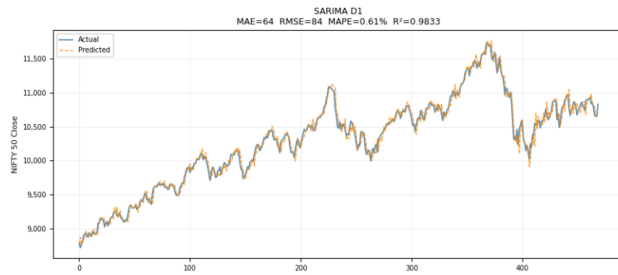
Dataset 2 (Full data, 2009 to 2024)

Post-Covid crash, and recovery markets were more volatile (only slightly). On this dataset LSTM which was trained on log returns achieved the lower MAPE and RMSE metrics. It was trained on 60 day sequences of returns, while refitting every 60 days to account for all the changes (walk forward approach) the LSTM model was able to track the actual shape of the market slightly better than other models.

Visualisations:

Consists of all the model's performances (even the failed ones) for Dataset 1 and Dataset 2 respectively.

Actual vs Predicted — NIFTY 50 | D1 — Pre-COVID (2009-2019)



Actual vs Predicted — NIFTY 50 | D2 — Full Period (2009-2024, incl. COVID)

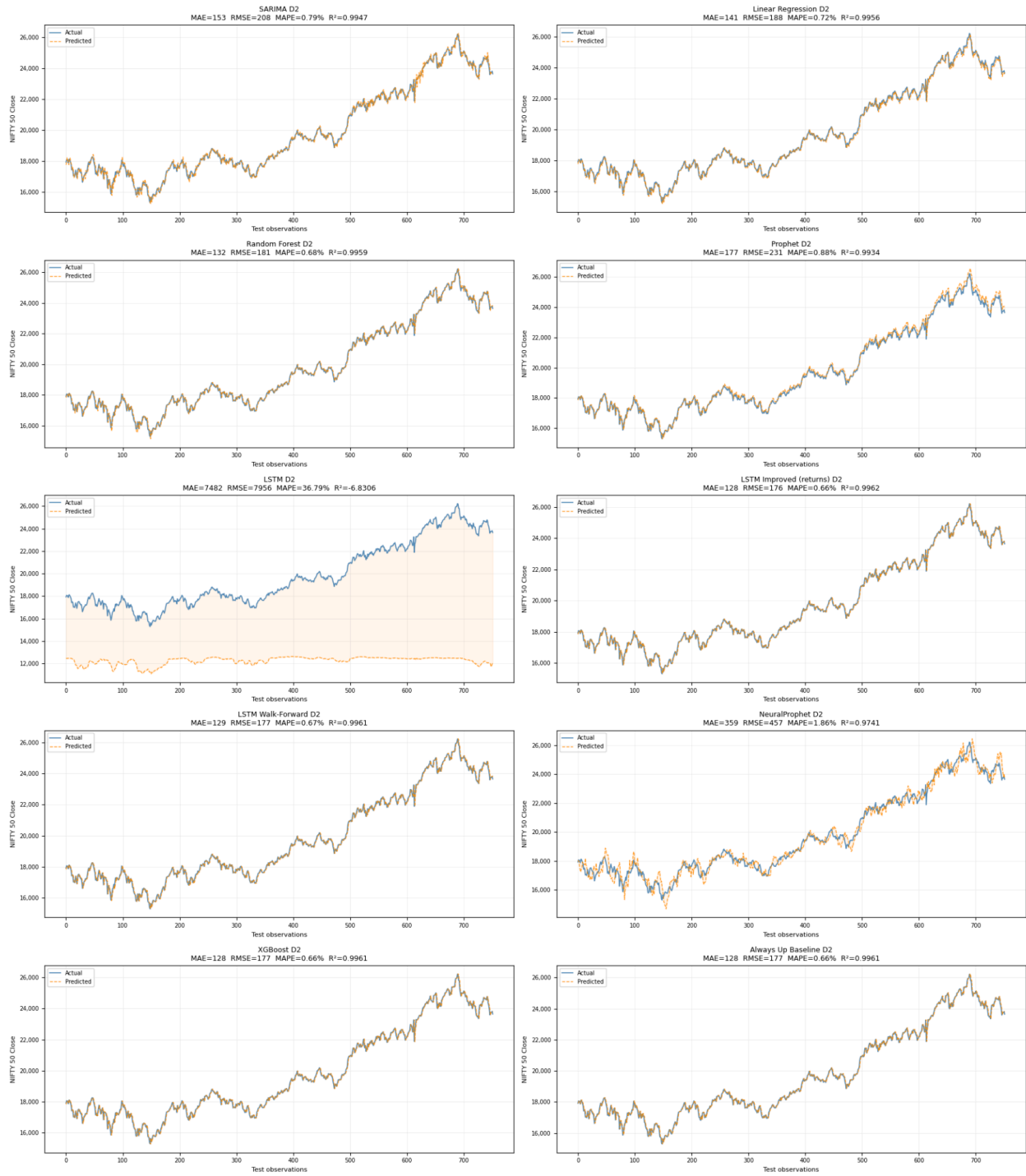


Figure (15): Performance of all models on D1 and D2
Actual vs. Predicted

Directional Accuracy

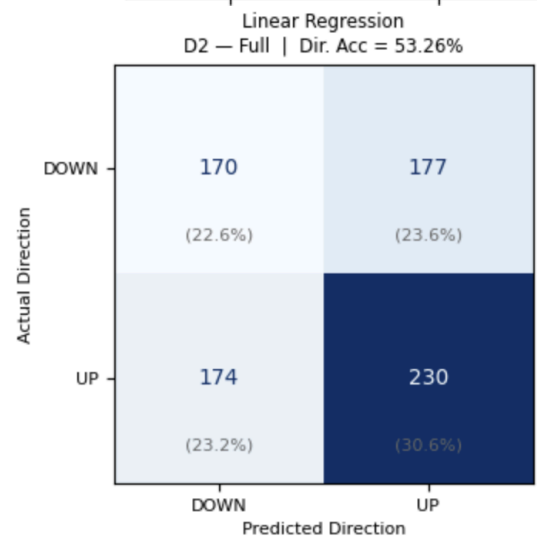
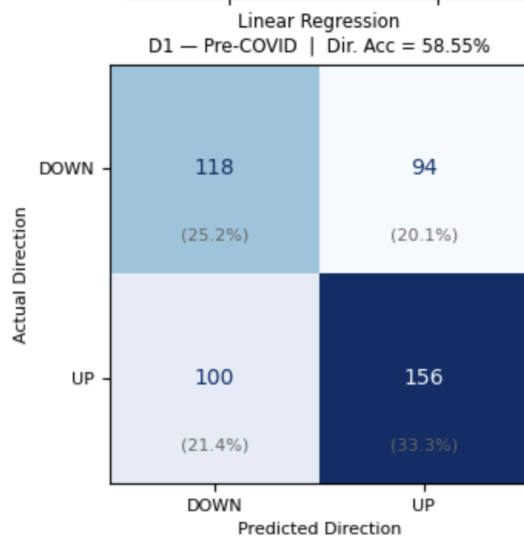
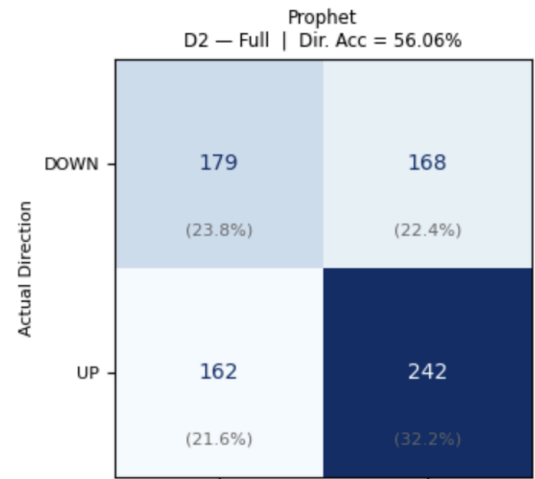
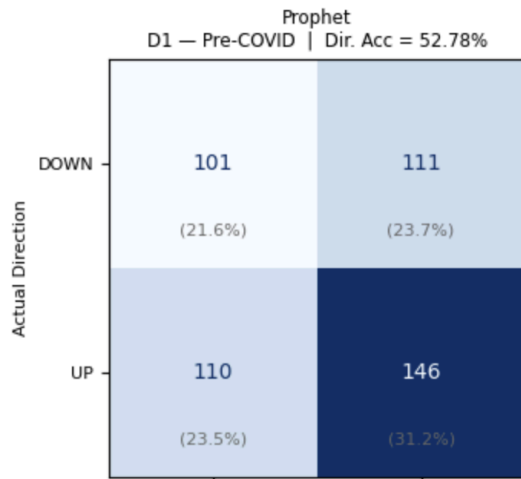
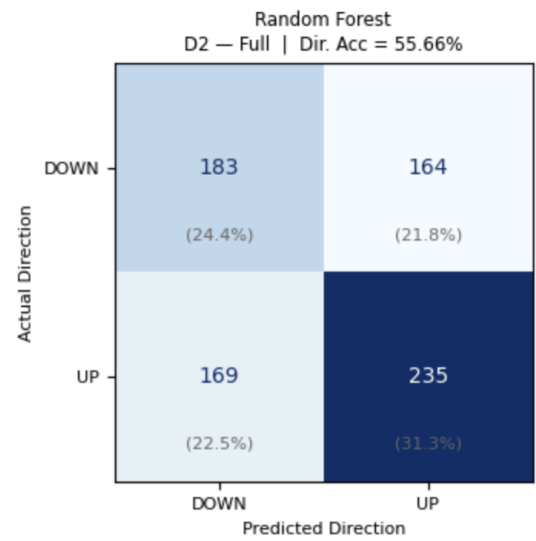
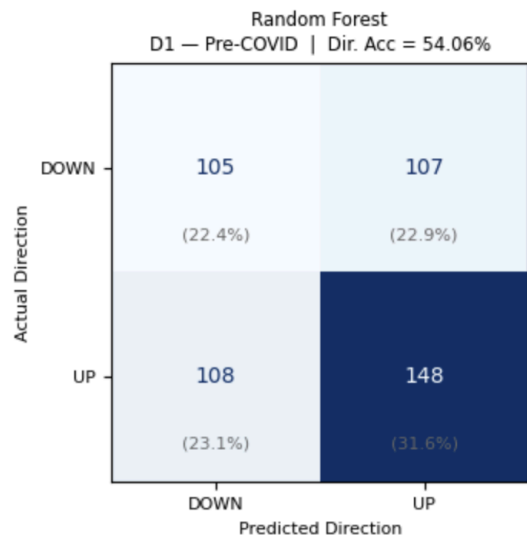
Directional accuracy is a classification problem with two outcomes: Did the market go up or down? A 50% score means the model did no better than a coin flip for predicting the market movement.

Note: The directional accuracy was derived POST-HOC. All the models were first used to predict the price or the log return and based on that prediction, I took the sign of change and got the directional accuracy. So the models were not exactly trained to classify the direction, it was derived only later.

Model	D1 Directional Accuracy	D2 Directional Accuracy
Linear Regression	58.55% (best on D1)	53.26%
Prophet (log returns)	52.78%	56.06% (best on D2)
Random Forest (log returns)	54.06%	55.66%
XGBoost (log returns)	53.42%	55.39%
LSTM Improved (returns)	53.85%	55.13%
LSTM Walk-Forward	53.63%	55.13%
Always Up Baseline	53.85%	55.13%
SARIMA	52.56%	50.60%
NeuralProphet	50.00%	50.20%
Original LSTM (on raw prices)	49.57%	51.40%

Figure (16): Directional Accuracy

This part of the project is still a working section. It will require different training especially for ML and DL models, and also different macroeconomic indicators. I will continue to work on the classifications which is a project in itself.



Figure(17): Visualisation of confusion matrix (only for top performing models)

Final Results

Rank	Model	RMSE	MAE	MAPE (%)	R ²
1	LSTM Improved (returns)	74.8	56.6	0.548	0.9869
2	LSTM Walk-Forward	75.2	56.9	0.55	0.9868
3	XGBoost	75.3	57.1	0.552	0.9867
4	Random Forest	79.7	63.1	0.609	0.9851
5	SARIMA	84.5	63.6	0.615	0.9833
6	Prophet	98.6	72.4	0.695	0.9773
7	Linear Regression	102	82.9	0.803	0.9757
8	NeuralProphet	249.7	197.5	1.926	0.8542

Figure (19)
Dataset 1 — D1 (Pre-COVID: 2009–2019 daily data, 80/20 split)

Rank	Model	RMSE	MAE	MAPE (%)	R ²
1	LSTM Improved (returns)	176.2	128.4	0.664	0.9962
2	XGBoost	176.7	128.4	0.665	0.9961
3	Random Forest	181.3	131.6	0.68	0.9959
4	Linear Regression	188.4	141.2	0.722	0.9956
5	SARIMA	207.7	153.5	0.789	0.9947
6	Prophet	231.4	177.3	0.88	0.9934
7	NeuralProphet	457.3	358.7	1.856	0.9741

Figure (20)
Dataset 2 — D2 (Full period: 2009–2024 daily data, 80/20 split)

References:

Hamayel, M. J., & Owda, A. Y. (2021). A novel cryptocurrency price prediction model using GRU, LSTM and bi-LSTM machine learning algorithms. *AI*, 2(4), 477–496.

<https://doi.org/10.3390/ai2040030>

Sonkavde, G., Dharrao, D. S., Bongale, A. M., Deokate, S. T., Doreswamy, D., & Bhat, S. K. (2023). Forecasting stock market prices using machine learning and deep learning models: A systematic review, performance analysis and discussion of implications. *International Journal of Financial Studies*, 11(3), 94. <https://doi.org/10.3390/ijfs11030094>

Hiransha, M., Gopalakrishnan, E. A., Menon, V. K., & Soman, K. P. (2018). NSE stock market prediction using deep-learning models. *Procedia Computer Science*, 132, 1351–1362.

<https://doi.org/10.1016/j.procs.2018.05.050>

Code:  Finance_Nifty 50_Final Project.ipynb

Please note this is a very messy code file and I will try to clean this.